Lecture 14 - Adaptive Query Processing

Setup

- The Postgres/Exodus/Starburst era was winding down
- Some fraction of the DB community was looking at XML databases
- Eventually there was emergence of MapReduce and NoSQL
- In between there was a bit of a lull
- One direction that some folks in the community started thinking about:
  - how to build query engines for networked, less-governed compute

- Examples
  - Queries that hit remote data sources
  - Streaming data
  - Peer-to-peer systems
  - User-driven experiences with interactive/approximate queries
  - Other systems settings with data needs (e.g., compiler analysis, declarative networking)
- Some themes that started to emerge in academic community
  - Asynchrony and unpredictable delay
    - “Satisficing” semantics:
    - Consumer apps like Web search and Napster changed expectations of correctness
  - Early/partial query results, ranking of answers
  - Weak consistency
    - Data integration on the fly
    - Security/Privacy
- This movement was in tension with the BSP batch processing model of MapReduce
  - No streaming, high latency, non-interactive
  - As it happened, MR captured popular interest in both OSS and academia
  - Rise of “Big Data” movement

Papers

- Eddies paper
- This survey - we'll look at some of this first...

Miscellaneous Background 1: Ordering Predicates

- Work by Hellerstein and then followed up by Surajit
- If you have a collection of predicates to be applied to tuples of a relation R, each with a certain cost c_i and probability of each predicate satisfying (independently) p_i
- Q: How do we order these predicates?
  - Order by $c_i/(1-p_i)$
- What if the predicates are not independent
  - Greedily order by $c_i/(1-p_i|p_{1\ldots(i-1)})$
  - NP hard

Algorithm 2.1 The Greedy algorithm for correlated selection ordering.

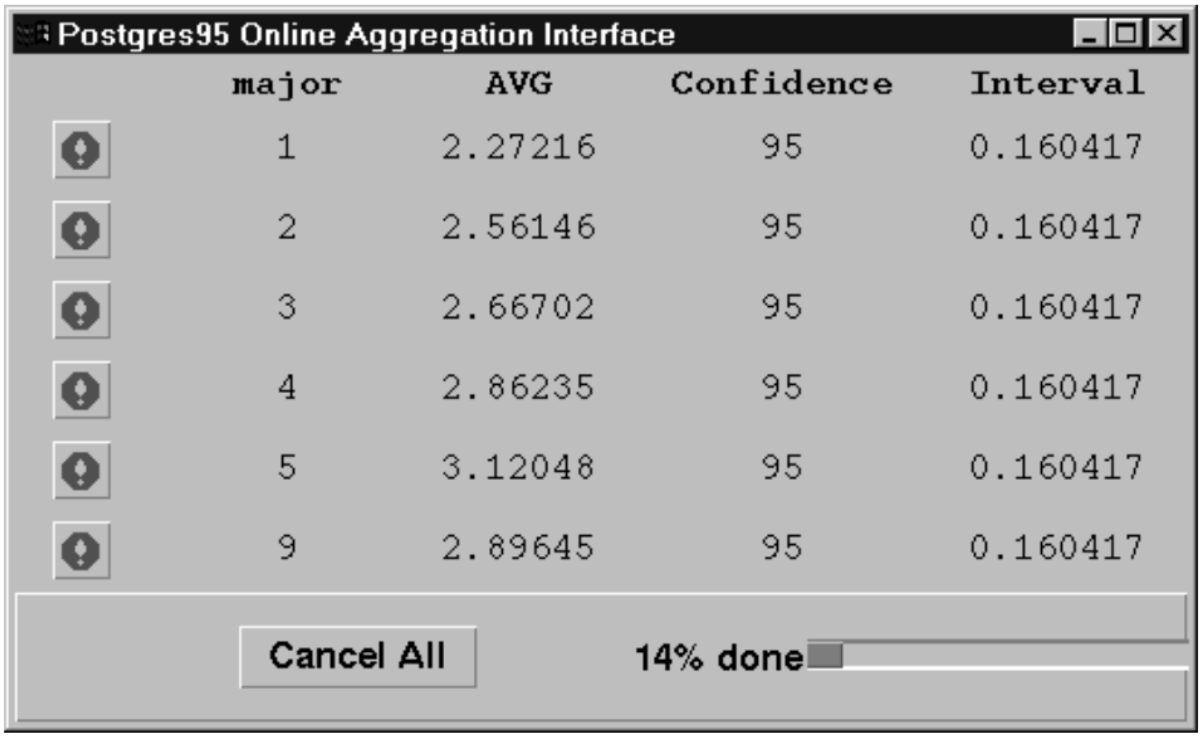
Input: A set of predicates, $S_i, i = 1, \ldots, n$; a procedure to compute *conditional probabilities.*

Output: A serial plan, $S_{\pi_1}, \ldots, S_{\pi_n}$, for applying the predicates to the tuples.

1. Choose S_{π_1} to be the predicate S_i with the lowest $c_i/(1 - p(S_i))$ among all predicates.
2. Choose S_{π_2} to be the predicate S_j with the lowest $c_j/(1 - p(S_j|S_{\pi_1}))$ among *remaining* predicates; $p(S_j|S_{\pi_1})$ denotes the conditional probability of S_j being true, given that S_{π_1} is true.
3. Choose S_{π_3} to be the predicate S_k with the lowest $c_k/(1 - p(S_k|S_{\pi_1}, S_{\pi_2}))$ among *remaining* predicates.
4. Continue until no operators left.

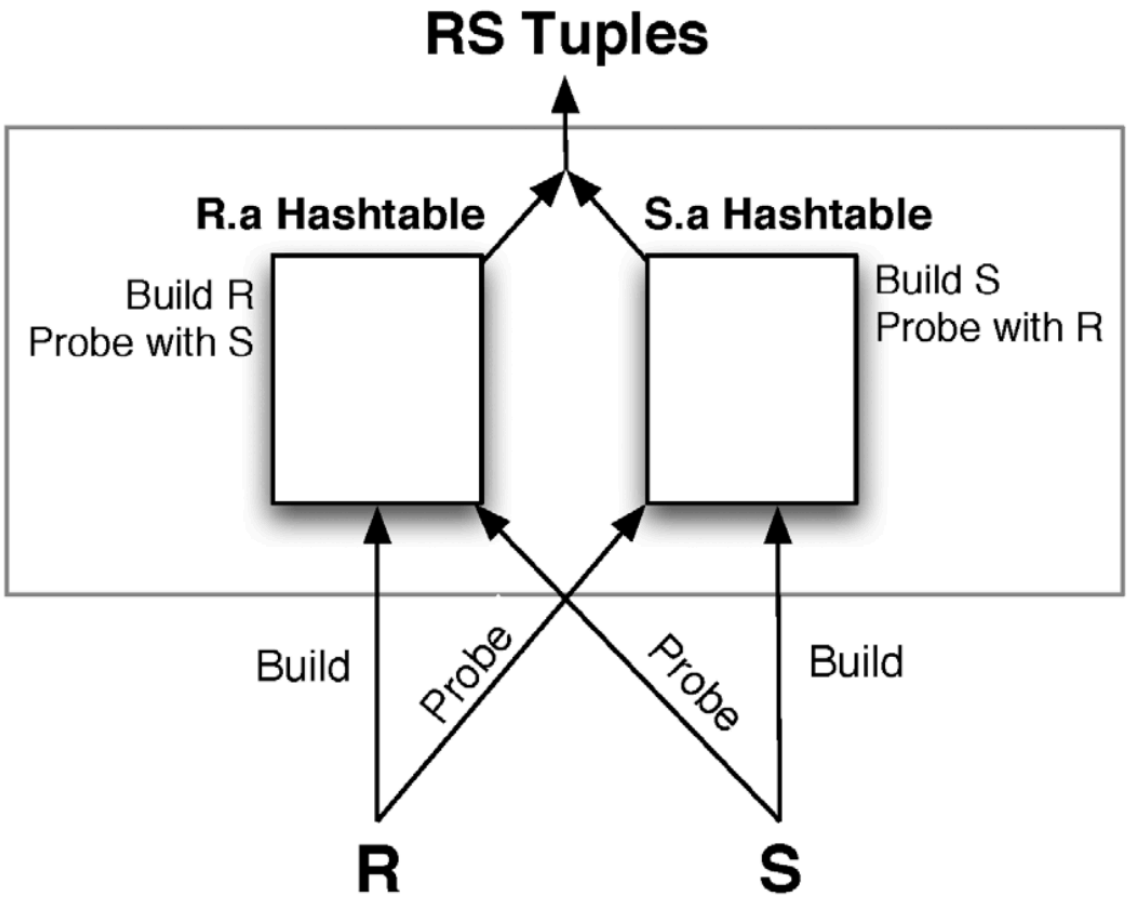
Miscellaneous Background 2: Online Aggregation

- Online Aggregation
- `SELECT AVG(sales), state from R GROUP BY state`
- You don't need perfect results; just get CIs
- Basic idea: boils down to samples per state
- Q: what is the expected AVG?
- Q: what are confidence intervals?
- Q: How do you guarantee randomness?
- Q: What about joins?



Miscellaneous Background 3: Symmetric Hash Join

- Q: How do I build a join operator that is not blocking?
- Pipelined Symmetric Hash Join



- Downside: more memory
- Ripple join is basically a generalization; adapts order in which tuples are read

Eddies

Setup

- Focus: textbook, Selinger-style Select-Project-Join queries.
- Concern: dynamic parameters for costing
  - Unpredictable selectivity for remote data
    - or stats may be stale or unavailable
    - can also vary during query processing
      - "*consider an employee table clustered by ascending age, and a selection salary > 100000; age and salary are often strongly correlated. Initially the selection will filter out most tuples delivered, but that selectivity rate will change as ever-older employees are scanned*."
  - Unpredictable delays in data sources
    - or hardware may be hard to predict
  - Unpredictable preferences from users (e.g. dynamic ranking/halting)
- Idea:
  - Adaptivity is helpful for query processing
    - *Why optimize -> execute, why not continuously reoptimize while executing*
  - Rather than a fixed dataflow graph, introduce a router called an “eddy”
  - Sits between all pairs of nodes and chooses “next hop” per tuple

- "*Eddies represent our first step in a larger attempt to do away with traditional optimizers entirely, in the hope of providing both run-time adaptivity and a reduction in code complexity.*"
- Architecture:
  - Inspired by Volcano's "exchange" operators, this is just another dataflow operator that encapsulates a (sub)graph of the dataflow
  - In principle can be added to any query engine and mixed/matched with traditional dataflow
  - Alternative: eddy as the primary scheduler for the whole query executor!
    - Easier to think about the control flow
  - Also inspired by Remzi Arpaci-Dusseau's River project
    - Shared-nothing dataflow that adapts to rate imbalances across machines
    - Used to win the Sort benchmark
    - Original goal was to combine River and Eddies for parallel adaptivity
  - Reorders pipelined operators on a tuple-by-tuple basis
  - n-ary tuple router between n data sources and k pipelined operators
  - can adapt tuple routing based on behavior when being routed to given operators

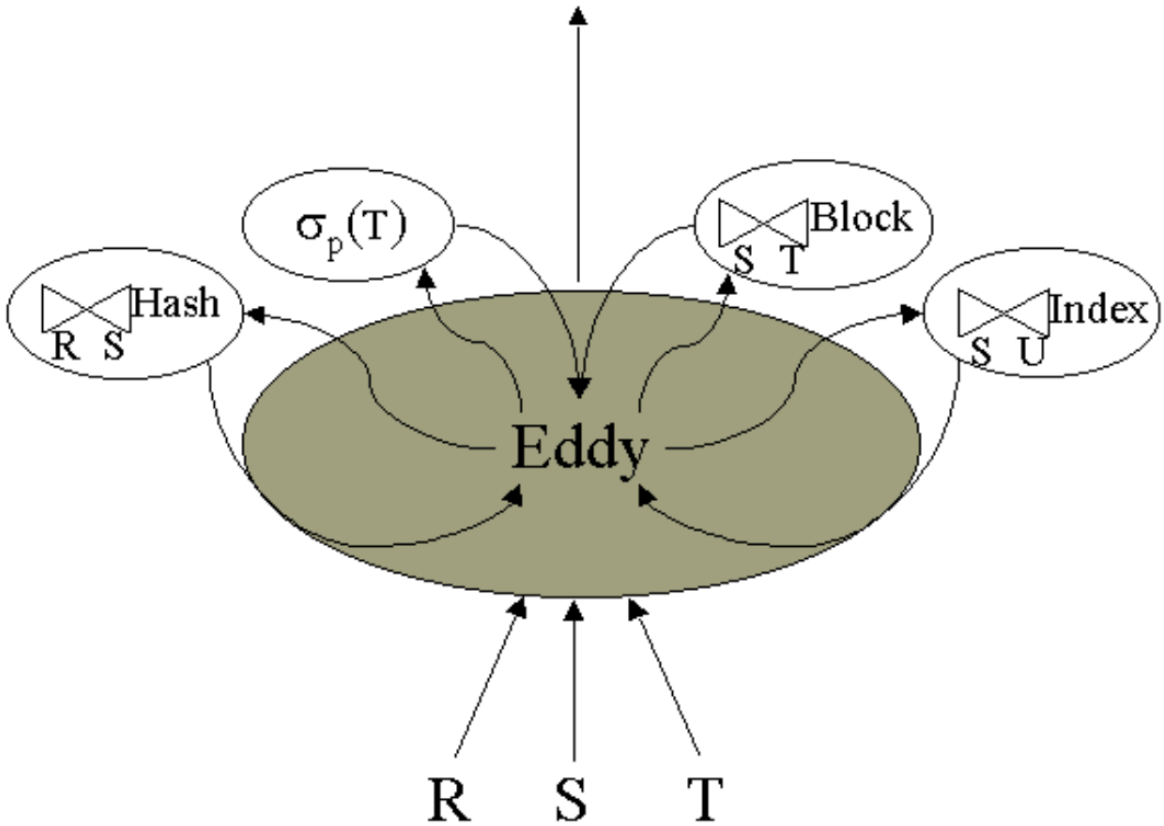


Figure 1: An eddy in a pipeline. Data flows into the eddy from input relations R, S and T. The eddy routes tuples to operators; the operators run as independent threads, returning tuples to the eddy. The eddy sends a tuple to the output only when it has been handled by all the operators. The eddy adaptively chooses an order to route each tuple through the operators.

- State is the Hard Part!
  - Decision-making cost vs. Adaptation cost
    - "As a philosophy, we favor adaptivity over best-case performance."

    - Identify “moments of symmetry” in query operators
        - *Join algorithms with frequent moments of symmetry and adaptive or non-existent barriers are thus especially attractive"*
    - Choose operators with frequent moments of symmetry to make the overhead of adaptation low
    - Avoid “Synchronization barriers” that prevent dynamic operator reordering (commutativity)
- Ripple Joins
    - Generalization of Pipelined/Symmetric Hash Join
    - Some state for each side allows frequent operator reordering

- Indexes and Remote Key Lookup Sources
    - Basically fancy unary operators, akin to “selection”
    - "An index nested-loops join is inherently asymmetric, since one input relation has been pre-indexed."
    - "*Even when indexes exist on both inputs, changing the choice of inner and outer relation “on the fly” is problematic" so best to think of "index join as a kind of unary selection operator on the unindexed input*"
    - Choice of indexes an other physical property enforcement? Not focus of current paper
        - "*indexes can be probed but not scanned, and hence cannot appear before their corresponding probing tables. Similar but more complex constraints can arise in preserving the ordered inputs to a merge join*"

Basic dataflow and state:

- Choice of operators in the plan is done by “pre-optimizer”
    - E.g. join algorithms, access methods
    - Eddy only reorders these
- To construct an output tuples, have to visit every join once
    - I.e. an output tuple has a complete provenance/lineage

- In-flight tuples have Done bits (lineage so far) and Ready bits (unfinished lineage)
    - The query dictates which Ready bits depend on which Done bits
- Eddy routes to operator inputs, operators route outputs back to eddy
    - Eddy tracks Ready/Done bits

- Optimizations
    - Batch up tuples, keep bits per batch
    - Batch decision-making: every so often build one or more static “routing tables” to choose from

Operators that allow reordering

- focus on operators that allow adaptivity rather than those with best performance
    - may lead to suboptimal plans
    - assumption that in a highly varying envt there is no such thing as best
- merging two sorted relations has a synchronization barrier, which we don't like
    - e.g., merging retrieves next lowest value, but what if this comes from the slower source
    - but this operator is symmetric - no input is special
- for operators that aren't symmetric, there may still be barriers
    - e.g., in NL, outer loop has to wait until inner loop finishes
    - but then once that happens can be reordered

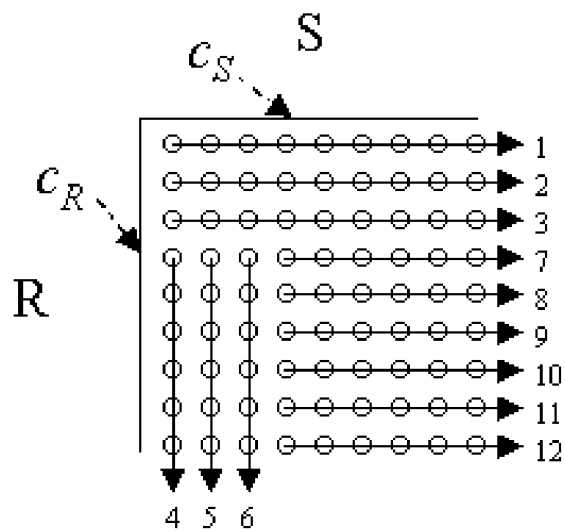


Figure 2: Tuples generated by a nested-loops join, reordered at two moments of symmetry. Each axis represents the tuples of the corresponding relation, in the order they are delivered by an access method. The dots represent tuples generated by the join, some of which may be eliminated by the join predicate. The numbers correspond to the barriers reached, in order. c_R and c_S are the cursor positions maintained by the corresponding inputs at the time of the reorderings.

- "*Some operators like the pipelined hash join have no barriers whatsoever*."
- Can extend idea to multiple relation join:
  - a cursor recorded for each input, and each loop would go from recorded cursor position to the end of the input.

Implementation

- Spanning tree of query graph where nodes represent relations and edges represent binary joins
  - Start with cartesian products across small tables, and then arbitrarily pick equijoin edges (on the assumption that they are relatively low selectivity), followed by as many arbitrary non-equijoin edges as required to complete a spanning tree.
- Along each equijoin edge can use either an index join if an index is available, or a hash ripple join.
- Along each non-equijoin edge it can use a block ripple join.